

Module 1

PostgreSQL Setup, Database & psql Basics

A 90-minute guided lesson with demonstration, hands-on practice and verification.

BUILD

01

working
database

PostgreSQL 18.4
Docker + psql

By the end, you can build and verify a working database

1 CONNECT

Start PostgreSQL and connect through psql using a host, port, user and database.

2 NAVIGATE

List databases and schemas, change database, inspect connection details and use built-in help.

3 CREATE

Create developer_lab, add the app schema and confirm the search path behaves as intended.

Success is visible: you can reconnect and reproduce the result without trainer assistance.

One continuous build turns concepts into working skills

01 Mental model

Server, client and database objects

15 min

02 Setup

Run PostgreSQL 18.4 in Docker

20 min

03 psql

Connect, navigate and get help

20 min

04 Hands-on

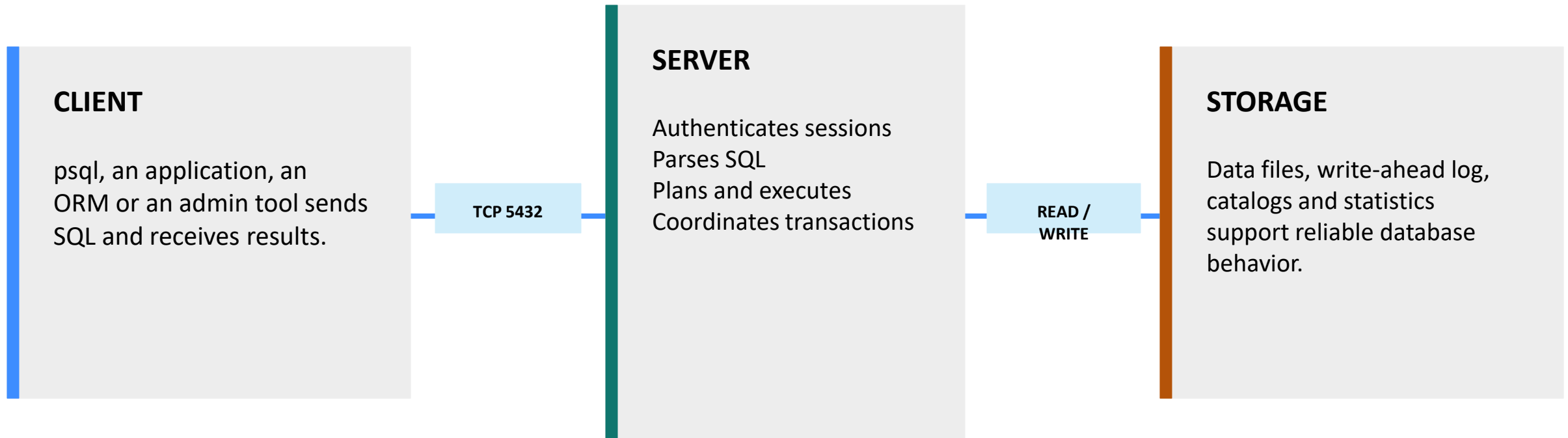
Create and verify developer_lab

30 min

Checkpoint

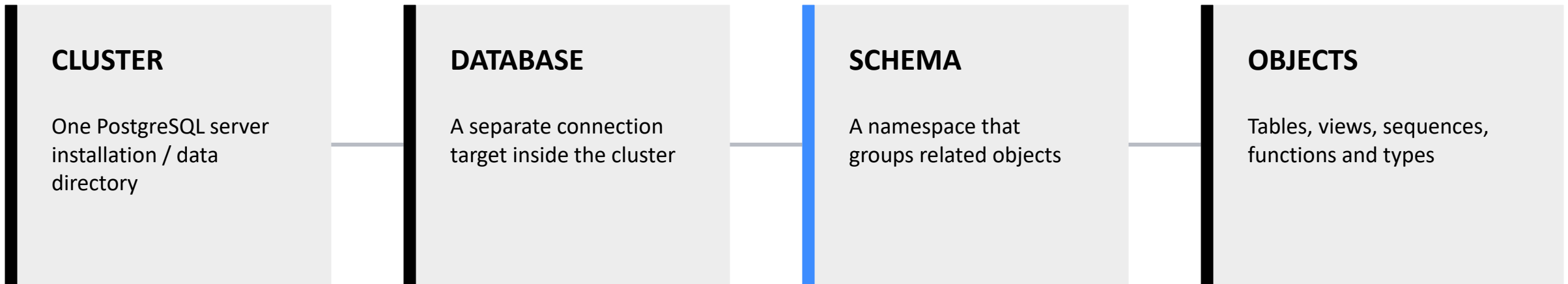
Explain what you created, where it lives and how your client reached it.

PostgreSQL serves every client request



Key idea: psql is the client; PostgreSQL is the server.

Know where each object lives before you create it



Example

pg-dev cluster → developer_lab database → app schema → future tables

A connection enters one database; schemas organize objects inside that database.

Use one predictable lab environment for the whole class

RECOMMENDED: DOCKER

- Same PostgreSQL version for everyone
- Easy start, stop and reset
- Minimal host configuration
- Named volume keeps lab data

ALTERNATIVE: LOCAL INSTALLER

- Works without Docker
- Uses a native service
- Paths differ by operating system
- Port or service conflicts are more likely

CLASS STANDARD

PostgreSQL 18.4 — current stable line; PostgreSQL 19 is still beta.

Pre-flight checks prevent most setup delays

SOFTWARE

- ☐ Docker Desktop or Docker Engine
- ☐ Terminal / PowerShell
- ☐ At least 1 GB free disk
- ☐ Port 5432 available

VERIFY

```
docker --version  
docker info
```

The second command must show an active Docker engine.

CLASS VALUES

Container: pg-dev
User: postgres
Password: postgres
Database: developer_lab
Schema: app

Training password only — never reuse this password in production.

Start PostgreSQL 18.4 with one repeatable command

```
docker run --name pg-dev \  
-e POSTGRES_PASSWORD=postgres \  
-p 5432:5432 \  
-v pg18data:/var/lib/postgresql \  
-d postgres:18.4
```

WHAT EACH OPTION DOES

- name identifies the container
- e initializes the admin password
- p publishes the server port
- v persists the data
- d runs in background

PostgreSQL 18+ volume target: /var/lib/postgresql

Bash / WSL shown. In PowerShell, run the command as one line.

Check readiness before attempting a connection

```
docker ps
```

```
docker logs pg-dev
```

```
docker exec pg-dev pg_isready -U postgres
```

EXPECTED SIGNALS

Container status shows “Up”

Logs end with “ready to accept connections”

pg_isready reports “accepting connections”

If readiness fails, diagnose the container first. A client cannot fix a server that is not running.

A connection needs four coordinates

HOST

localhost

Where the server runs

PORT

5432

Which network endpoint

USER

postgres

Who is connecting

DATABASE

postgres

Initial database target

```
psql -h localhost -p 5432 -U postgres -d postgres
```

No local psql? Use: `docker exec -it pg-dev psql -U postgres -d postgres`

psql meta-commands control the client

PSQL META-COMMANDS

Begin with a backslash

Do not end with semicolon

Examples: `\conninfo` `\l` `\dn` `\q`

They control navigation, display and client behavior.

SQL STATEMENTS

Use SQL keywords

Normally end with semicolon

Examples: `SELECT version();`
`CREATE DATABASE developer_lab;`

They are sent to PostgreSQL for execution.

Prompt changes help you detect unfinished SQL: `postgres=#` versus `postgres->`

Five psql commands are enough to navigate confidently

`\conninfo`

Show the current host, port, user and database

`\l`

List databases in the cluster

`\c developer_lab`

Reconnect to another database

`\dn`

List schemas in the current database

`\? / \h CREATE DATABASE`

Client help / SQL syntax help

`\q`

Exit psql safely

Create the database, reconnect, then create its namespace

```
\conninfo
\l

CREATE DATABASE developer_lab;
\c developer_lab

CREATE SCHEMA IF NOT EXISTS app;
\dn
```

WATCH THE CONTEXT

CREATE DATABASE runs in postgres. After \c, CREATE SCHEMA runs inside developer_lab.

VISIBLE RESULT

\l includes developer_lab.
\dn includes app and public.
\conninfo shows developer_lab.

search_path resolves unqualified object names

QUALIFIED NAME

app.products

Explicit and predictable. The schema is written before the object name.

UNQUALIFIED NAME

products

PostgreSQL searches schemas in search_path from left to right.

```
SET search_path TO app, public;  
SHOW search_path;  
SELECT current_schema();
```

Hands-on 1: build your own developer_lab environment

A	Start	Run the pg-dev container and prove the server is ready.
---	--------------	---

B	Connect	Enter psql and display your connection details.
---	----------------	---

C	Create	Create developer_lab and reconnect to it.
---	---------------	---

D	Organize	Create app and place it first in search_path.
---	-----------------	---

Do not copy a screenshot only—be ready to reproduce the commands.

Your success evidence should answer four questions

1 Am I connected?

\conninfo shows the expected user and database.

2 Does the database exist?

\l includes developer_lab.

3 Does the schema exist?

\dn includes app.

4 Will names resolve correctly?

SHOW search_path returns app, public.

Stretch: disconnect, reconnect directly to developer_lab, then repeat all four checks.

Troubleshoot from the server outward

CONTAINER NAME EXISTS

```
docker start pg-dev
```

Reuse the stopped container instead of running a duplicate.

PORT 5432 IN USE

```
Change -p 5433:5432
```

Connect with client port 5433.

PASSWORD FAILED

```
Check user and initialization
```

A reused volume keeps its original password.

SQL WILL NOT EXECUTE

```
\r or finish with ;
```

Clear an unfinished query buffer or complete the statement.

Knowledge check: explain the behavior

1

What is the difference between PostgreSQL and psql?

2

Why must you reconnect after CREATE DATABASE?

3

What does app, public mean in search_path?

4

Which evidence proves the lab succeeded?

Pair-share: 60 seconds each. Use one command as evidence in every answer.

You now have a working PostgreSQL foundation

CONNECT

Reach the correct server and database.

NAVIGATE

Inspect databases, schemas and context.

CREATE

Build `developer_lab` and the app namespace.

Next: Module 2 — Tables, data types, CRUD and constraints